# Beginner's Guide to Cyber & CTFs

Oct 22, 2026 · @Tom

Everything you need to get started: navigating the terminal, reading documentation, and the tools that show up in every CTF.

## Getting Started with the Terminal

The terminal (also called the shell, command line, or CLI) lets you talk directly to the operating system by typing commands. In CTFs, almost everything happens here.

**Opening a terminal**

- Linux: `Ctrl+Alt+T` or search for "Terminal" in your apps
- macOS: Applications → Utilities → Terminal (or search "Terminal" in Spotlight)
- Windows: Install Windows Subsystem for Linux (WSL) — then you have a full Linux terminal

**The prompt**

When you open a terminal you'll see something like:

```
user@hostname:~$
```

- `user` — your username
- `hostname` — the machine's name
- `~` — your current directory (`~` is shorthand for your home folder)
- `$` — means you're a regular user (a `#` means root/admin)

**Navigating the filesystem**

Think of the filesystem as a tree. You're always "in" one folder (your working directory).

| Command | What it does | Example |
|---|---|---|
| `pwd` | Print Working Directory — shows where you are | `pwd` → `/home/user` |
| `ls` | List files in current folder | `ls -la` shows hidden files + details |
| `cd` | Change Directory | `cd Downloads` · `cd ..` goes up one level |

| Command | What it does | Example |
|---|---|---|
| `mkdir` | Make a new folder | `mkdir ctf-work` |
| `cp` | Copy a file | `cp flag.txt backup.txt` |
| `mv` | Move or rename a file | `mv old.txt new.txt` |
| `rm` | Delete a file | `rm junk.txt` (no undo — careful!) |
| `cat` | Print file contents to screen | `cat flag.txt` |
| `less` | View a file one page at a time | `less bigfile.txt` (press `q` to quit) |

**Absolute vs relative paths**

- Absolute: starts from root `/` — e.g. `/home/user/ctf/flag.txt`
- Relative: starts from where you are now — e.g. `ctf/flag.txt` or `./flag.txt`
- `..` means "one folder up"; `./` means "right here"

**Running a program**

Type its name. If it's not in your PATH, prefix with `./`:

```
python3 solve.py
./my_binary
```

# Reading Documentation with man Pages

`man` (short for manual) is built into every Unix/Linux system. It's the definitive reference for every command — learn to use it and you'll never be stuck on "what does this flag do?"

**Basic usage**

```
man <command>
```

Examples:

```
man ls
man grep
man curl
```

This opens the manual in a pager. Navigate with:

- Arrow keys or `j` / `k` to scroll
- `/` to search within the page (then `n` for next match)
- `q` to quit

**Reading a man page**

Every man page follows the same structure:

| Section | What it contains |
|---|---|
| NAME | One-line description |
| SYNOPSIS | How to call the command (brackets = optional, CAPS = your input) |
| DESCRIPTION | Full explanation of what it does |
| OPTIONS | Every flag explained — the section you'll use most |
| EXAMPLES | Sample commands (not always present) |
| SEE ALSO | Related commands worth knowing |

**Searching for a command when you don't know its name**

```
apropos <keyword>
```

For example, `apropos base64` lists every command related to base64. Useful when you know *what* you want to do but not *which tool* does it.

**Quick flag lookups**

Most commands also accept `--help` or `-h` for a shorter summary:

```
grep --help
file --help
strings --help
```

Use `man` for complete reference; use `--help` when you just need to remember a flag quickly.

**`tldr` — community-written cheat sheets**

`tldr` is a community project that gives you practical examples instead of the full manual:

```
pip3 install tldr
tldr grep
tldr curl
```

Great for CTFs where you need the common usage pattern fast.

## Terminal Power Tips

These four habits will make you dramatically faster at the terminal.

**Tab completion**

Press `Tab` after typing the first few letters of a command, file, or folder name and the terminal fills it in. If there are multiple matches, press `Tab` twice to see all options.

```
cd Down[Tab]    →  cd Downloads/
cat mys[Tab]    →  cat mystery.bin
```

Always use tab completion — it prevents typos and is faster than typing.

**Command history**

- Press the **Up arrow** to cycle through previous commands
- `history` prints all past commands with numbers
- `!42` re-runs command number 42
- `Ctrl+R` starts a reverse search — type part of a command and it finds the last time you used it

**Pipes and redirection**

Pipes ( `|` ) connect commands: the output of one becomes the input of the next.

```
cat file.txt | grep FLAG
strings mystery.bin | grep -i flag
```

Redirection saves output to a file:

```
strings mystery.bin > results.txt      # save output
strings mystery.bin >> results.txt     # append to existing file
```

### Useful shortcuts

| Shortcut | What it does |
|---|---|
| `Ctrl+C` | Stop the running command |
| `Ctrl+L` | Clear the screen (same as `clear`) |
| `Ctrl+A` | Jump to start of line |
| `Ctrl+E` | Jump to end of line |
| `Ctrl+Z` | Suspend a process (bring back with `fg`) |

### Working with multiple files

Globs (`*`) match multiple files:

```
ls *.txt          # all .txt files
cat *.log         # print all .log files
file *            # run `file` on everything in the folder
```

## Common CTF Tools

These tools cover the vast majority of beginner and intermediate CTF challenges. Install the ones marked 📦 before a competition.

### Encoding & Decoding

| Tool | What it does | Example |
|---|---|---|
| `base64` | Encode/decode Base64 📦 built-in | `echo 'RVJST1I=' \| base64 -d` |
| `xxd` | Hex dump / hex decode 📦 built-in | `echo '464c4147' \| xxd -r -p` |
| `python3` | General-purpose scripting 📦 built-in | `python3 -c "print(bytes.fromhex('464c4147'))"` |
| `CyberChef` | Web-based multi-step decoder | gchq.github.io/CyberChef |
| ROT13 | Caesar cipher shortcut | `echo 'SYNT' \| tr 'A-Za-z' 'N-ZA-Mn-za-m'` |

### File Analysis

| Tool | What it does | Example |
|---|---|---|
| `file` | Identifies the true file type by magic bytes 📦 built-in | `file mystery.bin` |
| `strings` | Extracts printable text from any binary 📦 built-in | `strings mystery.bin \| grep -i flag` |
| `xxd` | View raw hex bytes of any file 📦 built-in | `xxd report.pdf \| head` |
| `binwalk` | Finds embedded files/archives inside a file | `binwalk firmware.bin` |
| `exiftool` | Reads metadata from images, PDFs, documents | `exiftool photo.jpg` |
| `unzip` | Extract ZIP archives 📦 built-in | `unzip archive.zip` |

### Web / HTTP

| Tool | What it does | Example |
|---|---|---|
| Browser DevTools | View source, headers, cookies, network requests | F12 in any browser |
| `curl` | Send HTTP requests from the terminal 📦 built-in | `curl -I https://target.com` (headers only) |
| Burp Suite | Intercept and modify HTTP requests | Community edition is free |
| `wget` | Download files from URLs 📦 built-in | `wget https://target.com/file.txt` |

### Cryptography

| Tool | What it does | Example |
|---|---|---|
| `openssl` | Swiss-army knife for crypto 📦 built-in | `openssl enc -d -aes-256-cbc -in enc.bin` |
| dcode.fr | Browser-based solver for 200+ ciphers | Great for classical ciphers |
| `hashcat` | Password / hash cracking | `hashcat -a 0 -m 0 hash.txt wordlist.txt` |
| `john` (John the Ripper) | Hash and password cracking | `john --wordlist=rockyou.txt hash.txt` |

### Forensics & Steganography

| Tool | What it does | Example |
|---|---|---|
| `steghide` | Hide/extract data in images and audio | `steghide extract -sf photo.jpg` |
| `zsteg` | Detect LSB steganography in PNG/BMP | `zsteg image.png` |
| `stegsolve` | GUI tool for image layer analysis | Java-based; run with `java -jar stegsolve.jar` |
| Autopsy / Sleuth Kit | Disk image forensics | For challenges with .img or .dd files |

**Networking**

| Tool | What it does | Example |
|---|---|---|
| `nc` (netcat) | Raw TCP/UDP connections 📦 built-in | `nc challenge.ctf.example 1337` |
| `nmap` | Port scanning and service detection | `nmap -sV target.com` |
| Wireshark | GUI packet capture and analysis | Open a .pcap file and filter by protocol |
| `tcpdump` | Command-line packet capture | `tcpdump -r capture.pcap 'tcp'` |

**Scripting & Automation**

| Tool | What it does | Example |
|---|---|---|
| `python3` + `pwntools` | CTF scripting toolkit for binary/web/crypto | `from pwn import *` |
| `jq` | Parse and query JSON on the command line | `cat data.json \| jq '.'` |
| `awk` / `sed` | Text manipulation | `awk '{print $2}' data.txt` |

# Quick Reference Cheat Sheet

The commands you'll reach for in every challenge.

**Navigation**

```
pwd                    # where am I?
ls -la                 # list all files (including hidden)
cd ~/ctf               # go to ctf folder in home directory
cd ..                  # go up one level
file *                 # identify type of every file in folder
```

### Reading files

```
cat flag.txt           # print entire file
less bigfile.txt       # page through a large file
strings binary.bin     # extract printable text
xxd binary.bin | head  # view first few lines as hex
```

### Searching

```
grep -r "FLAG" .            # search for FLAG in all files recursively
grep -i "flag" results.txt  # case-insensitive search
strings file | grep -i flag # filter strings output
find . -name "*.txt"        # find all .txt files
```

### Encoding / Decoding

```
echo 'RVJST1I=' | base64 -d                 # decode Base64
echo '464c4147' | xxd -r -p                 # decode hex
echo 'HELLO' | tr 'A-Za-z' 'N-ZA-Mn-za-m'    # ROT13
python3 -c "import base64; print(base64.b64decode('RVJST1I='))"  # Python decode
```

### Network / HTTP

```
nc challenge.ctf.example 1337        # connect to a CTF service
curl -I https://target.com           # fetch HTTP headers only
curl -v https://target.com           # verbose: show full request/response
curl -b "admin=true" https://target  # send a cookie
curl "https://target.com?id=1"       # send a URL parameter
```

### CTF workflow for a new challenge

1. `file <filename>` — what type is it really?
2. `strings <filename> | grep -i flag` — is the flag hiding in plain text?

3. `xxd <filename> | head` — check magic bytes
4. `exiftool <filename>` — any metadata?
5. Check the webpage's source code (browser: `Ctrl+U`)
6. Check HTTP response headers (`curl -I`)
7. Check cookies and URL parameters
8. Google the encoding format if you see suspicious-looking text